

From Spare Compute to Cumulative Mathematics

An open-source strategy for agent-assisted research on hard problems

WILL COOK*

September 2026

ABSTRACT

An open-source research environment lets contributors share the cost of navigation, experiment records, formal proof checking, claim review, and attribution. Once these tools exist, someone with a mathematical idea or a night of spare compute should be able to use them without rebuilding the infrastructure. This paper describes a strategy for doing so with agent-assisted mathematical research.

Contributors may supply compute, mathematical direction, infrastructure improvements, or review. Useful returns include proofs, counterexamples, finite computations, corrected statements, failed mechanisms, formal no-go theorems, and research tools. These can reduce the cost or improve the direction of later work when their interpretation survives review. Accepted mathematics enters a versioned problem corpus; accepted infrastructure changes alter how later work is produced and checked. The record preserves the origin of each contribution and the roles of those who supplied it.

The implementation is a public Lean repository organised around eight open Erdős problems, chosen to test the research process on difficult questions. A fresh clone contains papers, formal source, explicit open boundaries, corpus queries, validation programs, and structured return paths. All eight problems remain open. The repository demonstrates research infrastructure; it supplies no evidence that distributed agents outperform mathematicians or that more compute will solve a named problem. As of 31 August 2026, the author had recorded neither a completed external cold-clone use nor an accepted external contribution. Contributor experience therefore remains untested outside author-operated runs.

The account includes a production model, a contribution protocol with trust, security, and attribution rules, and a proposed evaluation. It compares the project with volunteer computing, Polymath-style collaboration, formalisation projects, and multi-agent proof systems. Mathematics is the first domain because its objects are digital and formal claims can be checked incrementally. Experimental science would require domain scientists, physical laboratories, safety governance, and other sources of evidence.

What is implemented

Contribution. A public contribution protocol accepts compute, mathematical direction, infrastructure, and review as independently useful inputs, while preserving evidence boundaries and role-specific credit. **Implementation.** One fresh clone contains eight problem worlds, formal source, explicit open obligations, validation programs, and structured return paths. **Limit.** All eight problems remain open, and no external contribution has yet tested the proposed contributor experience.

**Authorship and AI use.* Every word of this manuscript was generated by agents based on large language models operating within Will Cook’s private research system for artificial intelligence. Cook set the objectives and acceptance criteria, reviewed and authorised the manuscript, and is responsible for its contents. The agents are production tools, not authors. This production disclosure supplies no independent verification; Cook did not independently verify every mathematical claim.

1 The strategy

This project began with one undergraduate working without an institutional research team or a dedicated compute allocation. Those circumstances explain the interest in sharing infrastructure; they provide no validation of the mathematics or the software. The proposed research commons lets people share an environment while contributing different kinds of work.

A mathematician may supply a theorem, counterexample, reference, or correction. A Lean contributor may formalise or audit one statement. A person with spare compute may run an agent against a bounded public frontier; at present that means choosing a bounded task, operating it through a runner of their own, and returning evidence. There is no turnkey client. An infrastructure contributor may improve navigation, experiments, validation, reproducibility, or the public contribution path. A reviewer may compare a formal statement with its intended meaning or find an error in a returned result. The credit record should describe each contribution on its own terms, including work that leaves the problem open.

Contributors may use language models and agent harnesses throughout the work and must disclose that use. Model assistance alone says nothing about the mathematical value of a return. Acceptance concerns the attributable work: an idea or direction that produces useful mathematics, a proof or counterexample, a review, or an infrastructure change that improves later research. Each needs evidence and review appropriate to its claim.

The unresolved questions give the repository a continuing purpose. A run selects a subproblem, reads prior work, tests conjectures, records failed routes, formalises stable steps, and explains what was established and what remains open. Even when it produces no solution, it can leave a record that another researcher can inspect and use.

A *continuous goal* keeps the endpoint fixed across runs while updating the known results and remaining work. The next agent reads the accumulated theorems, computations, counterexamples, failed mechanisms, citations, and open obligations before choosing an attempt. Its return may be a theorem, a narrower reduction, a refuted route, a source correction, or a better question. The run is assessed by that return and its evidence. Preserving it avoids paying to reconstruct the same starting point.

Choosing what to automate

The choice of task should state what someone hopes to understand and what would change their next mathematical decision. Recovering an earlier proof, checking one calculation, comparing two mechanisms, or explaining a counterexample may be the useful task. A newly available solver is not by itself a reason to expand the search. When the purpose is learning, a hint or a check of the reader's own attempt may serve it better than the completed argument. The contributor guide and coupled-goals skill state this distinction; the reading guide offers an optional worked countermodel with the hint and explanation separated.

The September 2026 Math and AI declaration identifies conceptual understanding, the development of students and ideas, attribution, and human transmission as purposes that a race to solve problems can undermine [2]. For this project, that means choosing automation with the people doing the work and respecting the terms on which a question was shared. An open discussion of a student's project or an unfinished approach is not a request for an automated search to complete it. A bounded collaboration can still welcome substantial automation, but its purpose and expectations should be clear before the search begins.

For a substantial search, the record should preserve the details needed to interpret or reproduce it, including alternatives that explained a change of direction. Relevant details may include starting sources, tools, material human interventions, effort and the stopping condition; estimates and unknowns remain labelled. Small corrections need no run history. A reported success rate must include unsuccessful attempts. A timeout records a limit of that run; a mathematical obstruction requires an argument at its stated scope. These records can help compare methods under declared conditions. They do not establish a fixed boundary between problems that AI can and cannot solve, a distinction raised by Tao in his discussion of the changing difficulty landscape [3].

Discovery and stewardship

The proposed deployment separates two persistent roles. A *discovery goal* stays close to one mathematical frontier: it reads the corpus, compares attacks, uses computation to discriminate them, attempts ordinary and formal proofs, and returns the smallest stable theorem, counterexample, no-go, computation, or corrected boundary. A *stewardship goal* stays close to the corpus as a whole: it compares the new object with existing results, decides which declarations form one mathematical family, reconciles the papers and assurance records, and returns the next research question justified by that assessment. The corresponding systems contract is stated in the [coupled-goals section of the systems paper](#).

The two roles can be run by different people, agent harnesses, or machines. A compute contributor may operate only the discovery side. A mathematician may suggest a direction, assess whether a result is nontrivial, or repair its statement and exposition without paying for a long model run. An expositor can revise the paper, a Lean contributor can strengthen the formal interface, and an agent builder can reduce the cost of a run. The record credits the work each supplied.

Stewardship determines where future effort goes as well as how a result is presented. It may discover that a new “theorem” is only a reformulation, that five declarations are one coherent result family, that a short no-go eliminates an expensive research direction, or that a paper still leads with a weaker theorem. It then updates four separate outputs: proof and evidence status, mathematical appraisal, paper prominence, and the next allocation of compute or expert attention. These outputs may influence one another, but they are not one score and none can promote an unproved claim.

The roles resume when there is new work to assess. A stable mathematical change calls for stewardship; a revised assessment, an affected result that needs checking, or a sharper open boundary can justify another discovery run. When the repository is unchanged, neither role needs an agent turn to report that fact.

The public [run-coupled-research-goals skill](#) specifies how to run this sequence in a cold clone. It directs the mining and consequence-propagation jobs to use a shared source pin and exchange committed objects and receipts. One agent may alternate between the roles; different people, models, subscriptions, or machines may also perform them. The decisions remain separate in either arrangement.

The public clone must work without the private machine that produced the initial corpus. The private workbench is part of that production history and grants no proof or publication authority to a return. Contributors may use any model runner or no model at all, provided their work follows the public problem definition, evidence requirements, and review procedure.

The [systems paper](#) gives the complete claim-transition lifecycle. The [cold-clone paper](#) tests how a new agent reaches the public mathematics and the validation boundary. This paper describes participation, credit, and growth. The three accounts concern different parts of the same prototype.

What one clone lets a contributor do

A contributor does not need to understand the whole repository before doing useful work. From one clone, a person or agent can choose one of five first actions: mine a bounded problem route; contribute mathematical direction without paying for the compute that follows it; formalise or review one claim; repair the research machinery; or propose another sourced problem world. The clone-local [explain-public-system skill](#) lets an agent read the public corpus and companion papers on the newcomer’s behalf, explain the claim boundaries at the requested level, and point back to exact evidence. The reader can therefore begin with the ordinary request “explain this repository to me” rather than first mastering its file layout. The other skills select and run a frontier, coordinate the coupled goals, install the same workflows in a compatible agent harness, and describe the present multi-stage process for adding a problem.

The clone-local [task-entry command](#) makes that first choice inspectable. A request to “maintain public infrastructure” selects the maintenance workflow and its checks; a request to prove a remaining implication selects bounded research and points to the relevant corpus support and proof-plan queries. The output names the lane, a small set of documents to read, the next commands, and the claim boundary before work begins. These routes and their regression checks are implemented. Whether they reduce contributor effort or improve distributed research remains unmeasured.

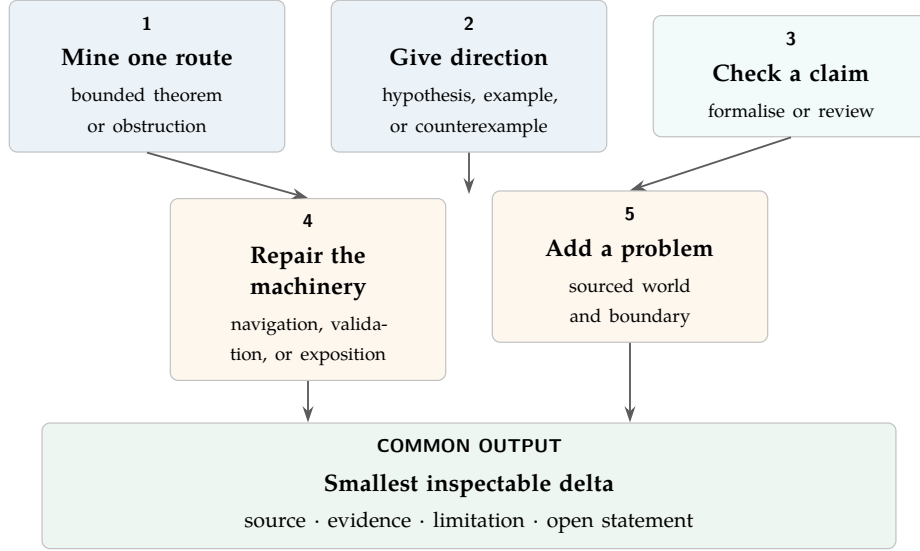


Figure 1: Five useful first actions. None requires an endpoint solution, and none receives a stronger status than the evidence returned with it.

2 A production model for mathematical progress

Candidate production depends jointly on compute, model capability, human direction, infrastructure, problem choice, and accumulated knowledge. Mathematical progress additionally depends on validation and scarce human review. The notation below makes those dependencies explicit. Let

$$C_t, A_t, H_t, I_t, P, K_t, V_t, R_t$$

denote, at time t , available compute, agent capability, contributed human mathematical direction, infrastructure quality, problem selection, accumulated research knowledge, validation capacity, and review capacity. The rate of candidate generation may be written schematically as

$$G_t = G(C_t, A_t, H_t, I_t, P, K_t). \quad (2.1)$$

The rate of *reviewable mathematical progress* is a different quantity:

$$Y_t = Q(G_t, V_t, R_t), \quad (2.2)$$

where Q includes formal checking, statement reconciliation, attribution, and editorial selection. Equations (2.1) and (2.2) are a conceptual production model, not a fitted empirical law. They record complementarity and bottlenecks. More compute can generate more attempts without increasing the rate at which strong results are validated or understood. A better model can still revisit a closed route if the corpus is difficult to navigate. Expert insight can change the search distribution without supplying any compute. Review capacity can be the binding constraint even when proof generation is cheap.

The open-source strategy adds two recursive updates:

$$\begin{aligned} K_{t+1} &= K_t + \Delta_t^{\text{theorem}} + \Delta_t^{\text{counterexample}} + \Delta_t^{\text{no-go}} + \Delta_t^{\text{computation}} + \Delta_t^{\text{correction}}, \\ I_{t+1} &= I_t + \Delta_t^{\text{navigation}} + \Delta_t^{\text{validation}} + \Delta_t^{\text{reproducibility}} + \Delta_t^{\text{governance}}. \end{aligned} \quad (2.3)$$

Only reviewed returns with explicit scope enter these updates. The first line retains mathematical results for later research; the second records changes intended to reduce cost or improve reliability. A failed attempt may supply a mathematical obstruction for K_t , or expose a recurring process defect whose repair belongs in I_t . Repairing the process does not change the truth status of a conjecture.

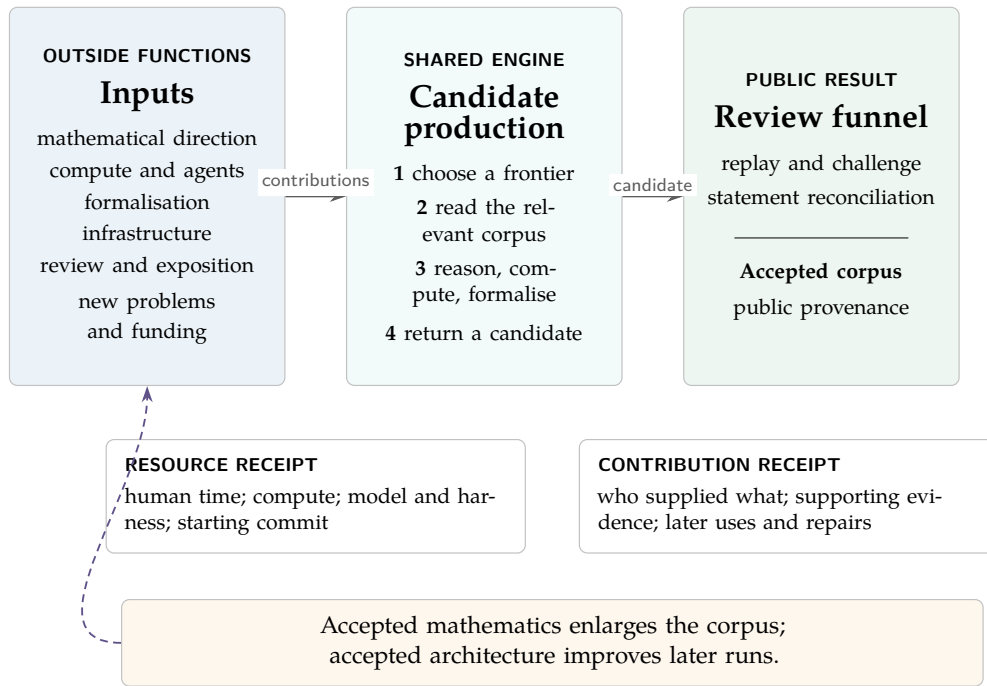


Figure 2: The conversion loop. The labels on the left are functions, not classes of people: one person may perform several, and several people may perform one. Search output crosses replay and review before it enters the accepted corpus. Mathematical returns enlarge the corpus; architecture returns alter the engine.

The model also gives a longitudinal use for the repository. A fixed, versioned problem world can be revisited as models, runners, and compute budgets change. Comparisons must disclose the starting commit, model, scaffolding, number of attempts, compute, and review procedure. Otherwise a success says little about which variable changed. A public failure record is equally important: capability claims are badly distorted when successes are announced and the number and cost of failed attempts are hidden [1].

3 Why begin with hard mathematics

Mathematics suits a first implementation because its objects, programs, papers, and formal statements can all travel in one clone. Many conjectures can be probed by exact computation. A proof assistant can check a stable formal step without waiting for an entire long argument to be written informally and retrofitted later. This does not make mathematical meaning automatic: Lean checks the proposition in the source, not whether it is the proposition the project intended to study.

Computation has a second role. It can provide an artificial form of local intuition to a system that is better at writing and running code than at reproducing a mathematician’s tacit judgement. Exact experiments can expose counterexamples, compare representations, and locate a narrow regularity. The results guide which approach to attempt and which conjectures to pursue. They leave the evidence class of an unbounded statement unchanged. In Bayesian language, an experiment can update which route deserves attention; it cannot assign a formal posterior probability to a theorem or replace proof.

The eight current problems were selected as demanding research environments, not as a claim that they were the eight most tractable or valuable open problems. Their papers expose materially different contribution points.

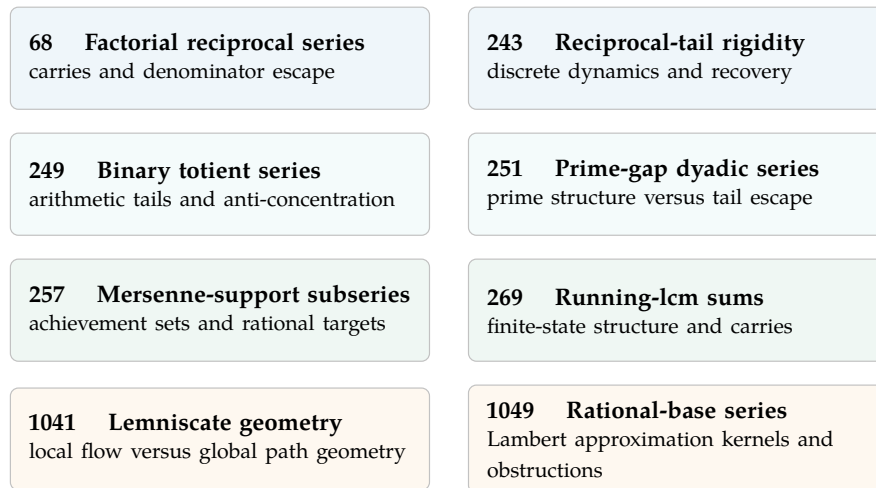


Figure 3: Eight distinct research environments. The results map and the problem papers state their exact frontiers; this strategy paper does not reproduce the specialist vocabulary needed to attack them.

Every row admits more than theorem proving. A mathematician may sharpen a hypothesis or supply a counterexample. A programmer may run a discriminating exact calculation. A formaliser may turn a paper deduction into a checked declaration or find that the two statements disagree. A literature reader may locate a theorem that closes or invalidates a route. Each return must state what was established and its limitation; the problem number supplies no additional evidence of value.

4 The public research object

The unit of participation is a *problem world*. A problem world includes the endpoint question, current public status, principal theorems, formal source, experiments, known counterexamples, failed mechanisms, source literature, and exact open obligations. A bounded task selects the part of that world needed for one question.

The research was initially developed inside a private workbench. The public release carries both the mathematical corpus and the workflows needed to inspect, validate, and return a contribution. A contributor uses the public checkout as the working environment; access to the private workbench is not part of the contribution protocol, and the workbench grants no proof or publication authority to a returned result.

Figure 4 separates the kinds of evidence and review available in the public project.

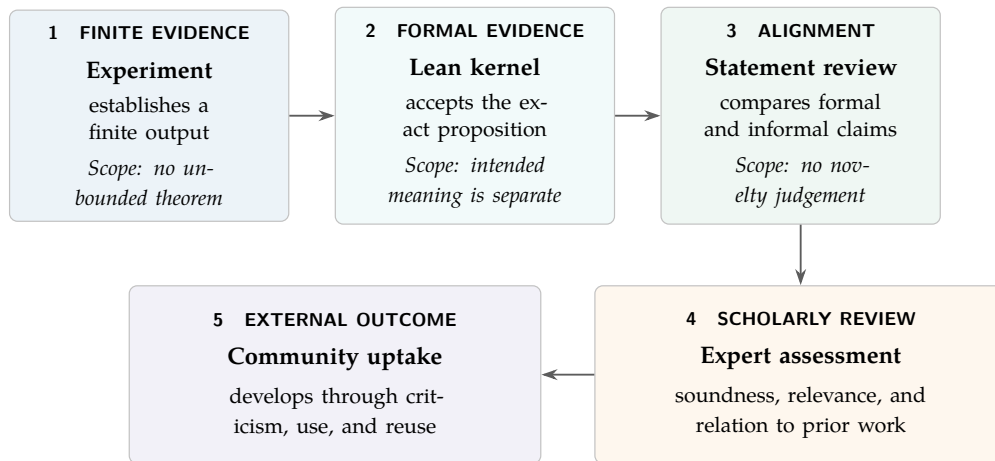


Figure 4: Evidence and acceptance remain separate. A candidate need not pass through every box in a single line, but no earlier box inherits the authority of a later one.

The project builds on established mathematical and computational practices. Erdős Problems supplies questions, sources, status, and a problem community [9]. Lean supplies the formal language and kernel [10]. DeepMind’s formal-conjectures supplies a public one-problem-file model with metadata, snapshots, issues, and pull requests [11]. Comparator and Palomar supply exact-interface checking, permitted-axiom inspection, and a durable formal record with a deliberately limited editorial claim [12]. Polymath supplies a social precedent for publishing small, tentative, and negative contributions [6]. BOINC and GIMPS supply the volunteer-compute precedent and make visible why executable work, independent checks, and legible credit matter [4, 5]. The present repository wraps these practices into a route that a newcomer can enter without first rebuilding each component.

The same sources can serve a short public primer, a specialist paper, a detailed proof account, Lean declarations, an agent explanation, and machine-readable queries. Every account must link to its source claims and evidence and preserve their scope, however briefly or informally it explains them.

An external runner begins at [the compact agent entry](#). It can inspect all problem frontiers, choose a bounded question, read the relevant paper and source neighbourhood, run experiments or edit formal code, validate the result, and prepare a return. None of these steps requires access to the private workbench. A contributor is free to replace the agent, the scheduler, or the entire search policy while retaining the public evidence boundary.

5 The contribution protocol

To propose a patch, a contributor forks or clones the repository, creates a branch, commits a focused change, pushes the branch to a public fork, and opens a pull request. A person who has a result but no patch can open a structured research-progress issue. A person with an infrastructure proposal can use the parallel architecture-proposal path.

The clone-local submit-pull-request skill gives agents the same procedure. It groups separable mathematical or infrastructure changes into coherent commits, runs the relevant checks, and prepares the repository’s pull-request template. A commit may span several files when they form one change. Pushing to a fork and opening the pull request are external actions and occur only after the contributor authorises them.

Upstream work may continue while the contributor’s branch remains open. The recorded starting commit supplies the common ancestor from which Git recovers the original delta; the contributor does not need a copy of the maintainer’s private or current working tree. Review therefore has two passes. First, reproduce the change in its original public context. Second, reconcile the accepted substance with current main and rerun the present checks. A material conflict resolution is a new, separately credited integration change. It does not erase or retrospectively rewrite the original contribution.

The complete protocol has nine steps.

1. **Clone.** Record the exact public commit and environment.
2. **Orient.** Read the claim boundary and the paper governing the selected problem or architecture area.
3. **Select.** Name one bounded frontier, expected evidence, and a stop condition. Working on all problems is allowed, but each returned result remains independently scoped.
4. **Work.** Use any mixture of reasoning, source research, computation, Lean, and human mathematical judgement. Preserve useful negative results instead of rewriting the run as a linear success.
5. **Validate.** Run the checker appropriate to the claim. A computation receives a computation receipt; a formal theorem receives a Lean receipt; neither receives semantic or publication authority from that fact alone.
6. **Propagate.** Enumerate the plausible Lean, claim, paper, computation, route, validation, and contributor consumers. Update each one, verify it unchanged, defer it with a re-entry condition, or explain why it is not a consequence. Repeat this pass after old-branch work is reconciled with current main.
7. **Return.** Submit source, evidence, starting commit, tool disclosure, collaborators, requested contribution roles, and the exact stronger statement that remains unproved.
8. **Review and adopt.** Reviewers reproduce the evidence, reconcile formal and informal statements, decide scope, and either accept, request revision, reject, or preserve the return as a scoped negative result.
9. **Publish and credit.** An accepted generation is linked to the paths and commit that assimilated it. Later corrections append lineage; they do not erase the earlier contributor.

A non-specialist compute contributor needs an intelligible task and a way to return evidence. They are not asked to certify a proof or invent a mathematically meaningful objective unaided. The default work unit is a packet prepared from an authored mathematical route: an exact open statement, relevant sources, a permitted experiment or proof obligation, an expected evidence class, and a stopping condition. Qualified mathematical review should precede large-scale distribution; it is not claimed for every present route. The contributor and agent execute the packet and return evidence. Lean checks a formal proposition. A qualified reviewer checks whether that proposition says what was intended and whether the return is relevant. The return remains a candidate until the applicable gates have been crossed.

The mathematician who supplies an input and the mathematician who verifies a return are also distinct roles. The first may contribute a conjectural mechanism, a counterexample pattern, or simply a well-posed question that causes the system to produce useful mathematics. The second inspects the resulting statement, argument, prior art, and significance. One person may perform both roles, but the provenance record must not let an input certify its own output merely because it came from a mathematician.

The protocol accepts two kinds of contribution. The *mathematics track* accepts proofs, reductions, computations, counterexamples, corrections, formalisation, and reproducible failed routes. The *architecture track* accepts improvements to agent workflow, navigation, validation, reproducibility, public experience, governance, and tooling. An architecture return never needs a fictitious problem number, and it cannot request promotion of a mathematical claim.

A pull request proposes a change; review records whether it is adopted, and a release records an immutable public version. These separate records permit credit for a useful counterexample, correction, or design even when no theorem is merged.

6 From an agent claim to mathematical attention

An agent saying “solution found” is an event in a run log, not a project claim. An informal derivation may contain an unnoticed gap. A Lean theorem may be completely kernel-correct while formalising the wrong statement, assuming an inadmissible axiom, or proving a result too weak to settle the problem. Formal verification therefore strengthens one layer of evidence; it does not collapse correctness, intended meaning, novelty, significance, and community acceptance into a single status.

The proposed review sequence reserves specialist attention for returns that have passed the earlier checks:

1. **Candidate.** A person or agent returns a scoped argument, computation, formal declaration, counterexample, or no-go.
2. **Replay.** Automated checks reproduce the code and attach the correct evidence class without promoting its claim.
3. **Internal challenge.** Independent agents or contributors try to break the statement, locate stronger prior art, vary the experiment, and reconcile formal and informal formulations.
4. **Maintainer triage.** A human checks whether the bundle is coherent and relevant, and whether the earlier checks justify asking a specialist to spend time on it.
5. **External record.** A mature Lean result may be packaged through Comparator and submitted to Palomar; a mathematical account relevant to a listed problem may be placed before the Erdős Problems community and the appropriate research audience.
6. **Acceptance.** Mathematicians inspect, explain, criticise, reuse, publish, or reject the work over time. Broad mathematical acceptance is a community judgement that the repository maintainer cannot grant.

Palomar mechanically verifies a pinned challenge–solution interface using Comparator and kernel checking, records structured disclosure, and applies a documented automated editorial filter. It explicitly does not endorse the result, establish novelty, or provide human expert review [12]. Likewise, the Erdős Problems forum asks long proofs or partial proofs to be linked as external documents and filters implausibly incomplete claims; it is a route to relevant attention, not a substitute for that community’s judgement [9].

Several contributors can make a return easier to assess through independent replay, adversarial review, repairs to different failure modes, a stable formal interface, or exposition that a specialist can audit. Their number alone provides no mathematical evidence. The work they record can justify specialist attention without strengthening the theorem’s status. This gives a contributor without institutional connections concrete reasons to ask for review, while sparing the expert the raw agent transcripts.

7 Progress, provenance, and credit

Each return states what kind of evidence it supplies. The accepted forms include:

- an unconditional theorem or a formal proof of an existing statement;
- a counterexample or corrected hypothesis;
- a no-go theorem excluding a defined strategy class;
- a bounded computation with code, range, and interpretation;
- a conditional reduction with its antecedent still visible;
- a reproducible failed route that prevents duplicated search;
- a literature correction or exact source locator;

- an architecture proposal or validated infrastructure change; or
- exposition or review that changes what another person can understand and check.

A single leaderboard would obscure the differences between compute volume, mathematical depth, software quality, correction work, exposition, and review. The public recognition view can list contributors with their accepted artefacts, evidence classes, problem or infrastructure area, and roles. The current receipt schema supports the fourteen CRediT roles as an optional vocabulary, including conceptualisation, methodology, software, validation, investigation, and writing [15]. CRediT describes work; it does not decide authorship.

Adopting a contribution does not give the project ownership of its underlying idea. The public record should name its originator and identify the work that was retained. Authorship and formal publication still require separate judgements. Recording roles preserves contributions that would disappear under sole solver credit or an undifferentiated collective name.

The same rule applies when a local result is carried into a broader library. The ledger should distinguish the person or run that produced the local observation from the expert who recognised its natural generality, reconciled it with prior art, designed the library interface, rewrote or formalised the proof, and stewarded review. If that expert turns a problem-specific result into a Mathlib-quality contribution, the project does not demand ownership of the resulting pull request or paper. It asks for a reasonable provenance link back to the route from which the idea arose, while crediting the expert fully for the mathematical judgement and upstream work they actually performed.

Credit for partial and negative work gives an outside contributor a reason to spend thought or compute on an unsolved problem. The same record makes later correction, reproduction, and literature attribution possible.

A contributor outside a university, research institute, or AI laboratory can point to the accepted artefact. Its receipt identifies their idea or implementation, the public files containing it, the evidence reviewers checked, their roles, and the remaining boundary. The record documents that contribution publicly. It confers no degree, peer-review status, employment recommendation, or guarantee of recognition by another institution.

Record now, trace consequences later

The first ledger should remain descriptive. A contribution receipt records who supplied which object, from which public state, under which role, with which evidence and limitation. It need not guess the contribution's eventual importance. Later accepted work can add reviewed lineage edges such as *uses*, *enables*, *repairs*, *formalises*, *refutes*, or *explains*. A consequence view can then show which later results, tools, or expositions depend on an earlier contribution.

Before returning a result, the contributor follows the clone-local propagate-research-consequences skill to inspect its plausible uses. Each affected statement, file, or process is updated, verified unchanged, deferred with a reason, or excluded from scope. Later work may reveal further consequences; known uses still need checking at the time of return.

These links provide a revisable account of later uses, without assigning an objective impact score. A later event does not establish causation, and an authored dependency claim can be mistaken. A small observation may have many consequences; a large compute run may have none beyond its negative result. Both keep their original receipt, and readers must be able to inspect and correct the subsequent links.

Any financial reward scheme would require a separate prospective policy for eligibility, conflicts, funding, tax, dispute, and the treatment of old receipts. Existing attribution creates no financial entitlement. A mining run can still disclose the model, provider, harness, compute donor, starting commit, and any human direction as distinct roles.

8 Running contributed code safely

BOINC packages scientific jobs for heterogeneous consumer devices, and BOINC Central explicitly aims to make volunteer computing available to scientists without the resources to operate their own project [4]. GIMPS shows a mathematical version: volunteers run a common search program, independent

machines confirm a prime, and discovery credit includes the compute donor, software authors, server operator, and wider volunteer effort [5].

Agent-assisted research is less uniform than either example. Most tasks are not interchangeable work units with a predetermined verifier. A runner may change code, propose a conjecture, or misread the problem. The public system therefore distributes *search* while keeping authority local to each evidence type. Mathematicians and formalisation contributors design or review the routes; volunteers may execute them with Claude Code, Codex, Cursor, Antigravity, OpenCode, another open or proprietary harness, or no agent at all. The shared task packet, evidence boundary, and return record permit different runners. Returned code is untrusted until reviewed. Expensive or privileged continuous-integration jobs must not execute fork code with repository secrets. In particular, a GitHub `pull_request_target` workflow must not check out and execute an untrusted fork; GitHub documents that combination as a route to secret leakage and repository compromise [16]. Initial checks should run with read-only permissions, no secrets, bounded resources, and explicit artefact retention. Promotion to trusted infrastructure is a later review decision.

A useful compute return records at least the source commit, model and runner, tool disclosure, prompt or task packet, wall-clock time, human time, token or subscription use, and hardware budget where available, together with commands, changed paths, outputs, validation receipts, failures, and the stopping rule. A mathematician's time, an API bill, and donated hardware cannot be reduced to one exchange rate. Reporting them separately still permits interpretable comparisons and makes the cost of the surrounding assistance visible.

The first public mode may remain deliberately simple: contributors clone the repository and run their own agents locally. A later volunteer-compute layer could distribute signed, immutable task packets and receive result bundles without granting write access. It should be built only after task identity, sandboxing, deduplication, resource budgets, result replay, and abuse handling have explicit owners.

9 Precedents

The strategy combines practices from Erdős Problems, Lean and mathlib, Comparator and Palomar, DeepMind's formal-conjectures, Polymath, BOINC, GIMPS, and recent multi-agent formalisation systems. Its intended contribution is to make these practices available through one clone.

Distributed compute. BOINC and GIMPS show that members of the public will donate hardware to a scientific objective when the client is easy to run, the work is visible, and credit is legible [4, 5]. Their tasks are much more mechanically uniform than open-ended proof research.

Distributed mathematical insight. Polymath projects invite participants at different mathematical levels to share small observations as they occur. Their rules explicitly welcome tentative and negative insights, provided they are made clear enough for others to absorb, and treat the project as collaboration rather than a race [6]. Polymath supplies a social protocol; it does not supply a formal proof, computation, and agent-return substrate for every comment.

Distributed formalisation. Mathlib uses ordinary fork-and-pull-request practice, human review, and source-level attribution. The Carleson formalisation used a public blueprint and many claimable lemma-sized tasks, with formalisation feeding corrections back into the informal plan [7, 8]. Google DeepMind's formal-conjectures repository similarly uses one-problem files, issue assignment, pull requests, metadata, and stable snapshots for a growing Lean benchmark [11]. These projects show how a large formalisation can be divided into tasks for public contributors.

Multi-agent formal research. Agent Hunt studies bounties, locks, guarded ownership, and collaborative agents for autoformalisation [13]. Lean Atlas uses formal dependency information to reduce the declarations a person must inspect for semantic verification [14]. These mechanisms address search, coordination, and review focus. The additional proposal here is a public adoption procedure that records the provenance of both mathematics and improvements to the research process.

Proof abundance. Tao separates problem solving into generation, verification, exposition, digestion and acceptance, and canonicalisation, and argues that proof abundance will create bottlenecks between these stages [1]. The strategy invites contributions at each of these stages, including review that prevents

an overclaim and exposition that explains a hard step. Tao’s rule of thumb, that a result whose authors cannot give a clear, correct, properly attributed expert-level account of it is incomplete even when formally verified, sets the standard for exposition here. A record designed to help a reader learn a result does not establish that its authors have supplied such an account. That must be shown for each result. At the time of writing, no selected result has a recorded expert account of this kind. Section 12 proposes testing whether the record helps people produce one.

A digestive-system analogy helps distinguish the preparation from its use: breaking down food is different from absorbing its nutrients. The AI system performs *pre-digestion* when it organises sources, expands a derivation, prepares examples, or connects a statement to its proof. People can do this preparatory work too. Mathematical digestion occurs when mathematicians work through the argument, understand why its assumptions and difficult steps are needed, and can explain and use its ideas. A readable paper shows that material has been prepared; it does not demonstrate human understanding. We use this analogy to express Tao’s separation of verification, exposition, and community digestion and acceptance [1, Sec. 6, pp. 6–8; Sec. 8, p. 11]. It describes the preparation for review. Claims still require verification, and human understanding must be demonstrated through what people can explain and use.

The implementation brings volunteer compute, small shared insights, formal task decomposition, problem worlds, recorded negative results, untrusted returns, credit by role, and human review into one open corpus. This combination is the proposed contribution. The paper makes no universal priority claim and reports no controlled comparison showing an increased discovery rate.

10 Adding another problem world

To extend the repository beyond the present eight problems, each new problem needs a canonical question and primary source; a dated account of its current status; an explicit public claim boundary; a readable primer or problem note; formal statements with an informal-faithfulness account where formalisation is appropriate; known literature, computations, counterexamples, and failed routes; exact open contribution points; navigation and validation routes; attribution; and a maintainer or review path.

The public skill for adding a problem separates three states. A proposal may begin as a sourced issue. An incubating formal lane may add checked source under a problem-owned namespace while stating that it is not yet a reviewed public claim. A fully indexed world joins the problem registry, paper corpus, query routes, return schema, cold-start inventory, and relevant external crosswalks. These are distinct transitions because a checked proposition, a published paper, a reviewed claim, and a Comparator selection are different facts.

After any of these transitions, consequence propagation inspects the problem registry, paper corpus, query routes, cold-start inventory, validation roster, return schema, and external crosswalks. A new problem is complete at its stated entry level only when the contributor has recorded what was done about each affected use.

The last transition is not yet a one-command public operation. The current paper-corpus fan-in has a maintainer-owned stage, several checks enumerate the present roster, and the bounded problem index is already close to its size limit. The immediate infrastructure work is therefore to derive rosters from one public authority, admit an explicit incubating state, split verbose detail out of the bounded index, and make an absent external crosswalk record a normal state rather than an error. The add-problem skill identifies these dependencies so that a contributor can account for them during integration.

The design aims to accommodate increasingly deep problem records, but its scaling has not been measured. A useful test asks whether a new agent can retrieve what it needs without reading every paper or rediscovering a known failure as the number and depth of problems grow. That test should be repeated whenever the roster or navigation machinery changes.

11 Participation and growth

The project should present several independent reasons to clone the repository.

- A **mathematician** can see eight exact frontiers and contribute a lemma, construction, counterexample, reference, or critique without learning the private orchestration system.

- A **Lean user** can formalise a paper deduction, audit a statement, simplify a proof, or improve the checked interface.
- A **compute hobbyist** can run a local agent against a bounded route in a real open problem and return reproducible evidence even when it is negative. The contributor is not asked to certify a proof claim; mathematical review remains downstream.
- An **agent builder** can compare runners on a stable problem world while disclosing compute, attempts, and starting state.
- An **infrastructure contributor** can improve the conversion from all other inputs to useful mathematical output.
- A **reviewer or expositor** can reconcile meaning, rank results, correct attribution, or turn a checked proof into recoverable understanding.

The first screen of the README should give the basic invitation: clone the project, point a local agent at the compact entry file, and ask it to choose a bounded frontier that matches the available tools. The same screen must say that all eight problems remain open, that a useful return need not solve one, and that infrastructure contributions receive credit.

The contribution procedure should work before the project is publicised through Hacker News, a research talk, or a model-community post. The claim would then be specific: an outside contributor can clone the corpus, find an exact open question, run or improve the tools, submit a result with its evidence, and find accepted work in the public record. Stars, clone counts, and agent-hours measure attention or activity, without establishing mathematical progress.

12 Evaluation

Evaluation should examine what happens between choosing a task, producing a return, reviewing it, and using the result. Initial measures include:

- time from a cold clone to a correctly scoped first task;
- fraction of returned bundles that can be replayed at the stated commit;
- fraction requiring statement or evidence-class correction;
- time from return to first substantive review and to adoption decision;
- repeated-dead-end rate before and after a no-go enters the corpus;
- reviewer time per accepted result family;
- contribution diversity across mathematics, computation, software, validation, exposition, and review;
- correction lineage completeness and attribution retention; and
- change in useful output at controlled compute when navigation or another infrastructure component changes.

Counts need mathematical interpretation. One accepted counterexample may be more useful than hundreds of generated lemmas. Generated certificate shards must not be counted as independent discoveries. A theorem accepted by Lean but rejected on intended meaning is not a successful public transition. Likewise, a strong negative result can improve the corpus even though the endpoint problem remains open. A retained attempt is not yet knowledge: a failed route may rest on a mistaken diagnosis, duplicate known work, or add nothing actionable. It counts only through a justified interpretation, such as a counterexample, a scoped obstruction, a reproducible stopping point, or a specific correction to the next task, and the record separates what happened from what was learned.

Longitudinal model comparisons should use immutable problem snapshots and held-out variants where possible. Public hard problems are susceptible to training-data contamination, and the repository itself will become more informative over time. A later run may therefore use both a stronger model and a more informative corpus. The study should measure these factors separately before attributing later success to the model.

This paper reports the implementation and protocol and proposes measures for a later study. It supplies no such evaluation.

A small first study should diagnose difficulties in the contribution path; it would not estimate their frequency across a population. An outside researcher receives one problem world and is asked to recover its strongest established statement, explain the main idea, state what remains unproved, distinguish a genuine obstruction from an unsuccessful attempt, and then undertake one bounded task and return it in a form the maintainer can assess without substantial reconstruction. Three presentations of the same material should be compared: the source, papers, and records in an ordinary repository; an information-equivalent static briefing written with care; and the navigation and record surfaces described here. The second condition is what separates the effect of the mechanisms from the effect of a better introductory paragraph. Failures are recorded at their own level: a command that could not be run, a source that could not be located, an interpretation that was corrected, or a result judged uninteresting are four different findings.

That study should also ask what the reader can do after the session: reconstruct the decisive step without copying it, explain why a tempting alternative fails, locate the prior idea, or formulate a nearby question with a mathematical reason for asking it. Record the actual explanation, correction or follow-up use and the help the reader needed. A completed checklist, shorter reading time or a model's judgement that the prose is clear cannot substitute for those observations. Nor should these activities become a compulsory score for exploratory work. They are proposed ways to learn whether the record helps people think; this paper reports no such reader outcomes.

13 Learning from every run

The research memory has three levels. Problem-local memory records the exact objects that change the mathematical search: proved lemmas, counterexamples, finite data, failed mechanisms, and unresolved obligations. Reusable mathematical memory records results that have survived a separate generalisation and integration pass. General process memory records lessons that transfer: a navigation failure, an experiment pattern, a formalisation pitfall, a validation gap, or a review rule. A local theorem does not enter the second level merely because its variables have been renamed.

A continuing discovery run should:

1. re-read the current frontier and the relevant corpus neighbourhood;
2. check whether the proposed route, calculation, or failure is already recorded;
3. compare distinct attacks and run the cheapest exact computation that can distinguish or refute them;
4. carry out the warranted analytic and incremental Lean work;
5. attack the strongest candidate for dropped hypotheses, counterexamples, and stronger prior art;
6. return the smallest evidence-bearing delta with its starting state and surviving boundary; and
7. add problem-local knowledge while proposing reusable process lessons separately for review.

Stewardship should then:

1. wake on a stable theorem, counterexample, computation, authority change, paper correction, or review outcome rather than on a timer;
2. rebuild the relevant candidate universe from source authority, including stronger results absent from the present paper or Comparator roster;

3. group declarations into coherent mathematical families and distinguish a new mechanism from a renamed residual, routine corollary, or duplicate;
4. decide proof authority, mathematical significance, exposition order, and future work allocation separately;
5. reconcile the paper, Comparator interface, Palomar disposition, claim boundary, navigation, and contributor lineage wherever the result has a real consequence; and
6. return a source-pinned frontier update: the strongest surviving result, its hard step, the exact boundary, and the next discriminating question.

A changed assessment of the mathematics can warrant a different paper order. The strongest exact results and mechanisms should lead, while routine supporting material remains available with less space. Comparator checks exact interfaces and Palomar provides an external registration route. Stewardship may prepare and prioritise submissions to them; it cannot confer novelty, acceptance, or canonical status.

Subagents can divide literature reading, computation, proof search, formalisation, and adversarial review when their questions and evidence remain independent. The integrating agent must read and verify their returns, and each lane keeps its own starting state and stop condition. Every attempt needs the same evidence regardless of how many agents contributed.

The system calls the transfer of a local process lesson to a reusable rule *up-propagation*. A proposal states the evidence, comparable cases, intended scope, and cases to which it should not apply. Review determines whether it warrants a change to a general skill or route. Computational reasoning can likewise become a specialised skill with reusable experiment forms, but every new experiment still records its finite domain and interpretation.

13.1 The local-to-general track

A theorem developed while studying a hard problem may be useful elsewhere, even if that problem stays open. Preparing it for broader use requires a separate mathematical review:

1. identify the exact local result and retain its problem-specific proof and provenance;
2. separate the hypotheses genuinely used from the coordinates and names of the motivating problem;
3. search the literature and the target library for an existing theorem, collision, preferred abstraction, and likely downstream consumers;
4. state the natural general result, with the local theorem exhibited as a transparent special case rather than hidden by the abstraction;
5. falsify proposed weakenings, check the general proof independently, and test at least one other real use; and
6. ask a qualified mathematician and Lean contributor to decide whether the result is important, well-shaped, documented, maintainable, and ready for an upstream discussion.

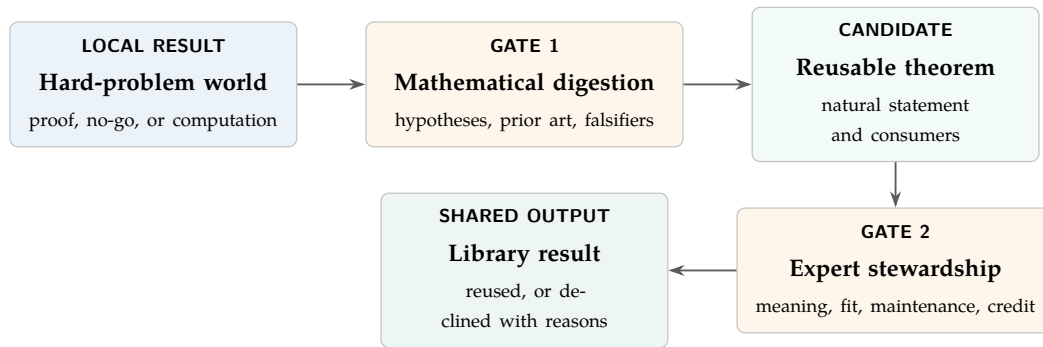


Figure 5: A local result becomes reusable only through a second mathematical and social gate. Formal correctness is necessary where Lean applies, but it does not decide generality, library fit, or stewardship.

Mathlib provides an external model for this final stage. Its contribution guide welcomes useful contributions and supplies a public fork-and-pull-request route; it does not reserve contribution to people with institutional credentials. It also sets high standards for generality, integration, maintainability, style, and documentation, and its current AI policy rejects low-quality unsupervised agent submissions while requiring AI use to be disclosed [7]. A compute donor should therefore not be directed to submit unreviewed agent lemmas to Mathlib. A subject and Lean expert must understand the candidate, decide whether it belongs, bring it to community standards, and take responsibility for the discussion. Credit follows the roles they performed. The originating problem remains part of the provenance and gives this project no claim to ownership of that contribution.

The present prototype does not automate this track and reports no Mathlib contribution produced by it. It can preserve local evidence and provenance, and it can help prepare a candidate. Canonical status and upstream acceptance remain external outcomes. The [systems paper’s mathematical loop](#) describes the authority separation underneath this handoff.

The research graph also records objects other than theorems. Nodes may be endpoints, conjectures, representations, computations, proofs, counterexamples, no-go theorems, reviews, or architecture changes. Edges distinguish proof dependency, implication, equivalence, refutation, obstruction of a method, experimental support, formalisation, correction, and publication. The edge type matters. A model-generated analogy can nominate a route; it cannot become a proof dependency or a public implication without the corresponding evidence.

As this graph grows, it may support a different kind of agent training or retrieval: successful proofs alongside tempting arguments paired with the exact reason they fail, and theorem statements alongside frontier neighbourhoods showing the remaining cut. Whether this improves mathematical originality or merely makes models more confident within known territory is an open empirical question.

14 A bounded transfer hypothesis beyond mathematics

The same abstract loop appears in experimental science: formulate a hypothesis, design an experiment, observe an outcome, update the theory, and retain both successful and failed interventions. A future agent might define a simulation or experimental script whose variables are linked to a formal model and a semantic account of the hypothesis. Physical observations could then update the research graph and select the next experiment.

The mathematical implementation tests this sequence of research operations. It supplies no evidence that the current system can conduct physics, chemistry, or biology. Formal verification has a special strength in mathematics because the target objects and proof rules are digital. In an experimental domain, the instrument, calibration, sample, protocol, measurement uncertainty, physical environment, and causal interpretation become independent authorities. A formally verified control program does not validate the scientific hypothesis or make a laboratory safe.

Any transfer would therefore require domain scientists, controlled physical infrastructure, ethics and safety review, material and environmental limits, and explicit human authority over experiment

selection and execution. The initial scope should be low-risk simulation or analysis of existing public data. Autonomous physical experimentation is not a present project goal.

15 Limits and governance

All eight endpoint problems remain open. The repository contains intermediate mathematics and deep records of failed routes, but no evidence that a distributed run will solve any problem. The claim that infrastructure improves the conversion from compute to mathematics has not yet been measured in a controlled experiment.

The prototype had no recorded external user or accepted outside contribution by 31 August 2026. Its clone instructions, agent skills, cross-paper links, pull-request path, and credit machinery may therefore contain friction that an author-operated audit does not reveal. Reproducing a clean clone, reporting a broken instruction, simplifying setup, repairing an unsafe default, or making the contribution route easier to understand are first-class architecture contributions even when they do not alter any mathematics.

The project remains centred on its maintainer. The initial problem selection, architecture, claim boundaries, review decisions, and public presentation were all chosen through one operator-controlled process. Open source makes those choices inspectable and contestable; it does not make them independent. A serious growth phase needs external maintainers and reviewers, published conflict and appeal rules, and a way for contributor-controlled reports to remain visible when the project declines to adopt them.

Review capacity is limited. More public agents may worsen the burden by generating plausible but low-value returns. Entry tests, evidence schemas, automated replay, and Palomar-style selection should protect expert attention, but automatic filters cannot award significance or community acceptance.

Contributors may dispute credit. A role vocabulary preserves more information than a single score, but it cannot mechanically decide authorship, priority, or the relative weight of an idea and its proof. The project must disclose its policy in advance and preserve correction history.

An open corpus can be gamed. Contributors may optimise for visible activity, generated theorem counts, or a public model comparison. Strong models may have encountered repository text during training. Security bugs may arrive as apparently useful workflow improvements. No automatic leaderboard should be allowed to become the objective function.

Open access also leaves resource inequalities in place. Compute donors, frontier-model providers, established mathematicians, and maintainers have different access and influence. The strategy can lower the fixed cost of entry and make attribution more durable. It cannot by itself make research participation equitable.

16 Execution order

Further implementation should follow this order.

1. Make the README state the experiment, contributor types, exact open boundary, clone command, agent prompt, return paths, and credit policy.
2. Keep every problem world independently navigable from a cold clone, with one paper-level open section and bounded query routes.
3. Accept both mathematical and architecture returns through typed schemas, human review, immutable generations, and public provenance views.
4. Provide runner-neutral local instructions and safe, unprivileged checks before building a hosted or volunteer-compute scheduler.
5. Recruit external reviewers and maintainers before increasing agent throughput substantially.
6. Run controlled studies of navigation changes, model changes, and compute changes against immutable snapshots.

7. Only after these boundaries work should the project consider a public task market, donated compute service, or transfer to another scientific domain.

The first three steps are partly implemented in the current public clone. Turnkey multi-provider mining, a public volunteer-compute scheduler, independent governance, and a cross-domain laboratory interface are not reported capabilities.

17 The next test

The public clone provides a route from a hard question to a bounded task, with procedures for returning evidence and crediting accepted work. Contributors can share the infrastructure while supplying mathematics, compute, formalisation, software, review, or exposition. Accepted results extend the corpus; accepted infrastructure changes revise how later work is produced and checked. Both keep their provenance.

The eight Erdős problems give this design real mathematical questions and exact open boundaries. Their difficulty requires the record to account for failed approaches, while leaving room to study changes in models, compute, infrastructure, and human direction over time. The next test is whether outside contributors can use the procedure, reviewers can assess their returns, and the accumulated record improves subsequent attempts.

A Public entry routes

The public repository is wcook04/plectis-erdos. The shortest current routes are:

Question	Public route
What is the experiment?	README.md
Where should an agent begin?	AGENTS.md
How can an agent explain the system?	explain-public-system skill
How can an agent run the coupled research lifecycle?	coupled-goal skill
How can an agent mine a frontier?	mine-open-problem skill
How is a bounded Lean change validated?	lean-concurrent-validation skill
How are downstream consequences reconciled?	propagate-research-consequences skill
How are clone skills installed elsewhere?	install-clone-skills skill
What is proved and what remains open?	docs/RESULTS.md
Which papers and problems exist?	docs/papers/README.md
How can I contribute mathematics?	CONTRIBUTING.md and the research-progress issue form
How can I improve the architecture?	architecture contribution guide and the architecture-proposal issue form
How can an agent prepare a pull request?	submit-pull-request skill
How can I propose or add another problem?	add-open-problem skill
How is a return validated?	erdos-research-return skill and the research-commons protocol
How is credit recorded?	credit policy
What do Comparator and Palomar establish?	Comparator guide and Palomar qualification

Declaration of generative AI use

Every word of this manuscript was generated by agents based on large language models operating within Will Cook’s private research system for artificial intelligence. The formal proofs and repository software were likewise drafted and revised by the agents through that system under Cook’s direction. Cook set the objectives and acceptance criteria, selected and reviewed the public claims, and approved the published version. Cook assumes responsibility for the accuracy, interpretation, and presentation of the work. Generative systems are production tools, not authors, and supply no independent authority.

References

- [1] T. Tao, *Mathematics in the age of AI*, 2026, [arXiv:2608.16753](#).
- [2] Math and AI, *A Severe Misalignment of AI in Mathematics*, [declaration](#), accessed 12 September 2026.
- [3] T. Tao, *Discussion of the difficulty landscape for mathematical problems*, Mathstodon post, 8 September 2026, [part 3 of 4](#), accessed 12 September 2026.

- [4] D. P. Anderson, *BOINC: A Platform for Volunteer Computing*, Journal of Grid Computing 18 (2020), 99–122, [DOI](#).
- [5] Great Internet Mersenne Prime Search, *GIMPS*, project documentation and discovery-credit record, [mersenne.org](#), accessed August 2026.
- [6] Polymath Project, *General polymath rules*, [project rules](#), accessed August 2026.
- [7] Lean community, *Contributing to mathlib*, [contributor guide](#), accessed August 2026.
- [8] L. Becker et al., *A Blueprint for the Formalization of Carleson’s Theorem on Convergence of Fourier Series*, 2025, [arXiv:2405.06423](#).
- [9] T. F. Bloom, *Erdős Problems*, problem database, sources, and forum, [erdosproblems.com](#), accessed August 2026.
- [10] L. de Moura and S. Ullrich, *The Lean 4 Theorem Prover and Programming Language*, Automated Deduction—CADE 28 (2021), 625–635, [DOI](#).
- [11] Google DeepMind, *formal-conjectures*, [software repository](#), accessed August 2026.
- [12] Palomar Registry, *About Palomar and Contribution policy*, [registry documentation](#) and [submission standard](#), accessed August 2026.
- [13] C. E. Brown, C. Kaliszyk, and J. Urban, *Agent Hunt: Bounty Based Collaborative Autoformalization With LLM Agents*, 2026, [arXiv:2603.06737](#).
- [14] B. Yanahama and A. Sannai, *Lean Atlas: An Integrated Proof Environment for Scalable Human–AI Collaborative Formalization*, 2026, [arXiv:2604.16347](#).
- [15] NISO, *CRedit: Contributor Roles Taxonomy*, [role definitions](#), accessed August 2026.
- [16] GitHub, *Preventing pwn requests*, GitHub Actions security guidance, [documentation](#), accessed August 2026.